FeedbackIQ — GCP Deployment Runbook

Recommendation: Compute Engine, not Cloud Run. Your `docker-compose.yml` already runs the whole stack correctly on a machine with a normal filesystem — it mounts `data/` and `models/` as host volumes. A Compute Engine VM *is* that kind of machine, so this path needs zero code changes. Cloud Run has no persistent disk, so it would mean adding Cloud Storage FUSE volume mounts and moving your API/Groq keys into Secret Manager — more moving parts, and a slow first request (reading a 1.35GB index over FUSE) unless you pay for `--min-instances=1` anyway, which erases most of Cloud Run's cost advantage for an app that's up most of the time. If your traffic is genuinely bursty/rare later, Cloud Run is worth revisiting — but Compute Engine is the right starting point.

I'm assuming you're starting from scratch (no GCP project yet, gcloud not installed) since that wasn't confirmed — skip to **Phase 2** if you already have a project with billing enabled and gcloud working.

Rough cost: the `e2-standard-4` (4 vCPU / 16GB RAM) size below runs about **$100–140/month** if left on 24/7. See the **Cost control** section at the end for how to stop it when you're not using it.

Phase 0 — GCP account, project, billing

1. If you don't have a Google Cloud account: go to https://console.cloud.google.com, sign in with a Google account, accept the terms. New accounts get $300 in free trial credit.
2. Create a project (via console is easiest for a first project): console → project picker → **New Project** → name it e.g. `feedbackiq` → note the **Project ID** it generates (not the display name — you'll need the ID).
3. Enable billing on that project: console → **Billing** → link a billing account (create one if needed, requires a payment method). GCP will not let you create a VM without this step.

Phase 1 — Install and configure gcloud CLI

On your Mac:

```
# Install (Homebrew is simplest)
brew install --cask google-cloud-sdk

# Or the official installer if you don't use Homebrew:
# curl https://sdk.cloud.google.com | bash && exec -l $SHELL

gcloud init
```

`gcloud init` will open a browser to log in and let you pick the project you just created. Confirm it took:

```
gcloud config get-value project
gcloud auth list
```

Enable the APIs this deployment needs:

```
gcloud services enable compute.googleapis.com storage.googleapis.com
```

Phase 2 — One-time local prep: precompute keywords, stage artifacts

Run these from your project root (`~/Desktop/FeedbackAnalytics_LLM`) with your local venv active.

```
        # 1. Generate the dashboard keyword file so the
VM doesn't have to
        #    (spaCy over ~26M words — this takes tens of
minutes, run it once)
        python scripts/precompute_keywords.py
```

```
        # 2. Create a bucket and push the ~3GB the app
actually needs at runtime.
        #    Pick a region close to where your users are
— europe-west2 (London)
        #    is just an example, swap it for whatever
fits you.
        gcloud storage buckets create gs://feedbackiq-
artifacts --location=europe-west2

        ./scripts/deploy/sync_artifacts.sh push
gs://feedbackiq-artifacts
```

That script moves exactly these 7 paths (confirmed against your repo):

Path	Size	Needed by
`data/embeddings/langchain_index/`	1.35 GB	RAG
`data/embeddings/reviews.faiss`	942 MB	semantic search

Path	Size	Needed by
`data/processed/reviews_unified.parquet`	407 MB	analytics, search metadata
`models/distilbert-finetuned-final/`	256 MB	sentiment
`models/classical/`	76 MB	TF-IDF + classical baselines
`data/results/`	6 MB	evaluation page
`data/processed/complaint_categories_all_negative.json`	32 KB	zero-shot categoriser

The raw corpora (~13GB), BERTopic models (~3.9GB), and `review_embeddings.npy` are training/discovery artifacts — the running app never touches them, so they don't get pushed.

Phase 3 — Create the VM and open the frontend port

```
        gcloud compute instances create feedbackiq \
          --machine-type=e2-standard-4 \
          --image-family=ubuntu-2204-lts --image-
project=ubuntu-os-cloud \
          --boot-disk-size=50GB \
          --scopes=storage-ro \
          --tags=feedbackiq

        # Only the frontend needs to face the internet —
the backend stays
        # reachable to the frontend over the docker-
compose network only.
        gcloud compute firewall-rules create feedbackiq-
ui \
          --allow=tcp:8501 --target-tags=feedbackiq
```

`e2-standard-4` (16GB RAM) matches what DEPLOY.md sized for — enough headroom to hold the FAISS index, the parquet file, and the transformer models resident in memory at once. Don't go smaller than this; the app will OOM.

Phase 4 — SSH in and bring the stack up

```
gcloud compute ssh feedbackiq
```

Everything below runs **on the VM**, once you're in:

```
sudo apt-get update && sudo apt-get install -y
docker.io docker-compose-v2 git
sudo usermod -aG docker "$USER" && newgrp docker

git clone <your-repo-url> FeedbackAnalytics_LLM
cd FeedbackAnalytics_LLM
./scripts/deploy/sync_artifacts.sh pull
gs://feedbackiq-artifacts

cat > .env <<'EOF'
ENVIRONMENT=production
GROQ_API_KEY=<your groq key>
API_KEY=<output of: python3 -c "import secrets;
print(secrets.token_urlsafe(32))">
LOG_LEVEL=WARNING
EOF
chmod 600 .env

docker compose up -d --build
docker compose logs -f backend   # wait for
"Warm-up complete", then Ctrl+C
```

A couple of things to get right here: - `ENVIRONMENT=production` makes the backend **refuse to start** unless `API_KEY` is a real generated value, not the dev default — that's intentional, not a bug you need to work around. - Generate the `API_KEY` value with the `secrets.token_urlsafe(32)` command shown above — don't reuse anything from `.env.example`. - `<your-repo-url>` — if your GitHub repo is private, `git clone` will prompt for auth; easiest is a GitHub personal access token used as the password when prompted, or set up SSH keys on the VM first.

Phase 5 — Verify

`http://<EXTERNAL_IP>:8501`

Get the external IP with:

```
gcloud compute instances describe feedbackiq --format='get(networkInterfaces[0].accessConfigs[0].natIP)'
```

You should see the Streamlit dashboard load. Try the chatbot page to confirm the Groq key and RAG index both loaded correctly — the backend logs (`docker compose logs -f backend`) will show any startup errors clearly if something's missing.

Before sharing this URL with anyone but yourself: it's plain HTTP on a raw IP right now. Put a reverse proxy with TLS in front of it — Caddy is the least fuss (auto-provisions a Let's Encrypt cert if you point a domain at the IP), or use a GCP HTTPS load balancer if you want it fully managed.

Redeploying after a code change

```
gcloud compute ssh feedbackiq
cd FeedbackAnalytics_LLM
git pull
docker compose up -d --build
```

If `data/` or `models/` changed too (e.g. you retrained something), re-run `sync_artifacts.sh push` locally first, then `sync_artifacts.sh pull` on the VM before rebuilding.

Cost control

The VM bills whether or not anyone's using the app. If this is for demoing rather than 24/7 use:

```
# Stop billing for compute (keeps the disk, so nothing is lost)
gcloud compute instances stop feedbackiq

# Start it again before a demo
gcloud compute instances start feedbackiq
docker compose up -d   # containers don't auto-restart on stop/start by default unless you added restart: always
```

To tear it down completely (irreversible — deletes the boot disk too):

```
        gcloud compute instances delete feedbackiq
        gcloud compute firewall-rules delete feedbackiq-
ui
```

The artifacts bucket (`gs://feedbackiq-artifacts`) is cheap to leave around (~$0.06/GB/month for the ~3GB you pushed) even if you delete the VM — useful if you want to redeploy later without re-uploading.

If you outgrow this and want Cloud Run later

Your `DEPLOY.md` already has the Cloud Run command (`gcloud run deploy` with a Cloud Storage FUSE volume mount and `--min-instances=1`), worth revisiting once you know your real traffic pattern. Not the right starting point today.